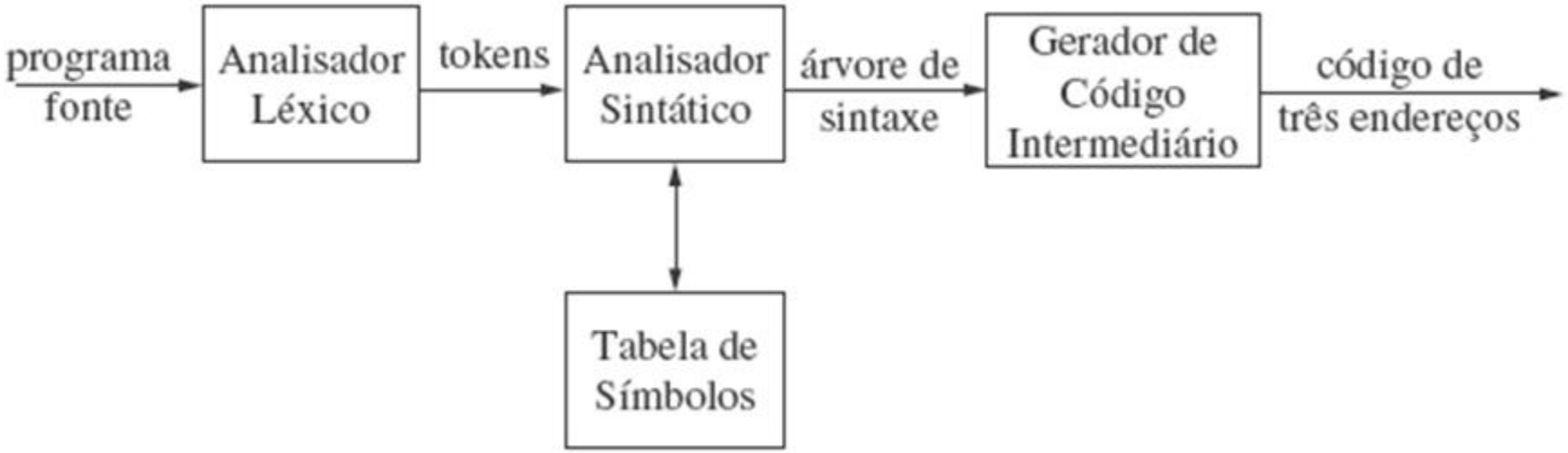
FIAP

Linguagens, Autômatos e Computabilidade - LAC

Prof. MSc. Fabio H. Pimentel
proffabio.pimentel@fiap.com.br

ANALISADOR LÉXICO – EXPERIMENTAL

Tradutor Dirigido por Sintaxe



Objetivos

Construir um Tradutor Dirigido por Sintaxe, na forma de um programa em linguagem C completo, que traduz um programa fonte em linguagem C para um fluxo de tokens e seus valores de atributos, e monta uma tabela de símbolos.

A Entrada deve ser um arquivo .txt com o programa fonte.

A Saída devem ser dois arquivos .txt, sendo 1 arquivo com o programa fonte traduzido em termos de tokens e valores de atributo, e outro arquivo com a Tabela de Símbolos.

Informações Complementares

O valor de atributo deve ser um ponteiro que aponta para um endereço na Tabela de Símbolos.

Através deste endereço uma estrutura de dados deve dar acesso a características específicas do token tais como:

a) lexema

b) token

c) padrão

d) ocorrências (linha x, coluna y)

A Tabela de Símbolos deve ser montada através de Alocação Dinâmica de Memória e estrutura de dados, formando uma lista encadeada.

Informações Complementares

Manipulação de arquivos em txt

Biblioteca: **#include<stdio.h>**

Informações Complementares

Manipulação de arquivos em txt

Biblioteca: `#include<stdio.h>`

Declaração global com ponteiro para o arquivo: `FILE *Arquivo;`

Informações Complementares

Manipulação de arquivos em txt

Biblioteca: `#include<stdio.h>`

Declaração global com ponteiro para o arquivo: `FILE *Arquivo;`

Abrindo/Criando um arquivo para gravação:

```
Arquivo = fopen("log.txt","w");
```

Informações Complementares

Manipulação de arquivos em txt

Biblioteca: `#include<stdio.h>`

Declaração global com ponteiro para o arquivo: `FILE *Arquivo;`

Abrindo/Criando um arquivo para gravação:

```
Arquivo = fopen("log.txt","w");
```

Fechando um arquivo: `fclose(Arquivo);`

Informações Complementares

Manipulação de arquivos em txt

Funções do Grupo 1

Para um caractere por vez.

Informações Complementares

Manipulação de arquivos em txt

Funções do Grupo 1

Para um caractere por vez.

Gravando um caractere no arquivo:

```
fputc(ch,Arquivo);
```

sendo ch a variável que armazena o caractere

Informações Complementares

Manipulação de arquivos em txt

Exemplo 1: armazenando frase de 1 linha em um arquivo (até digitar ENTER)

```
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>

int main( void )
{
        FILE *fptr;
        char ch;

        fptr = fopen(“arqtext.txt”,”w”);

        while ((ch = getche( )) != ‘\r’)
                fputc( ch , fptr );

        fclose (fptr);

        return 0;
}
```

Informações Complementares

Manipulação de arquivos em txt

Funções do Grupo 1

Para um caractere por vez.

Lendo um caractere de um arquivo:

```
ch = fgetc( Arquivo );
```

Informações Complementares

Manipulação de arquivos em txt

Exemplo 2: leitura de um arquivo, um caractere por vez e impressão no monitor

```
#include<stdio.h>
#include<stdlib.h>

int main( void )
{
        FILE *fptr;
        short int ch;

        fptr = fopen("arqtext.txt","r");

        while ((ch = fgetc(fptr)) != EOF)
                printf("%c",ch);

        fclose (fptr);

        return 0;
}
```

Informações Complementares

Manipulação de arquivos em txt

Exemplo 3:

é o exemplo 2 alterado com o feof( ):

A função feof retorna verdadeiro se o

final do arquivo for atingido.

Caso contrário, retorna falso.

```
#include<stdio.h>
#include<stdlib.h>

int main( void )
{
        FILE *fptr;
        short int ch;

        fptr = fopen("arqtext.txt","r");

        while ( ! feof(fptr))
        {
                ch = fgetc(fptr);
                printf("%c",ch);
        }

        fclose (fptr);

        return 0;
}
```

Informações Complementares

Manipulação de arquivos em txt

Exemplo 4:

é o exemplo 3 com teste de abertura:

```
#include<stdio.h>
#include<stdlib.h>

int main( void )
{
   FILE *fptr;
   short int ch;

   if((fptr = fopen("arqtext.txt","r")) == NULL)
   {
      printf("Não foi possível abrir o arquivo");
      exit(1);
   }

   while ( ! feof(fptr))
   {
       ch = fgetc(fptr);
       printf("%c",ch);
   }

   fclose (fptr);

   return 0;
}
```

A função fopen retorna um ponteiro com NULL caso o arquivo não pôde ser aberto.

Informações Complementares

Manipulação de arquivos em txt

Funções do Grupo 2

Para strings.

Informações Complementares

Manipulação de arquivos em txt

Funções do Grupo 2

Para strings.

Gravando uma string no arquivo: **fputs(“texto”, Arquivo);**

Informações Complementares

Manipulação de arquivos em txt

Exemplo 5: criando um arquivo texto e gravando uma string por vez

```
#include<stdio.h>
#include<stdlib.h>

int main( void )
{
        FILE *fptr;
        fptr = fopen("Testestr.txt","w");
        fputs("Se as coisas são inatingíveis... ora!\n", fptr);
        fputs("Não é motivo para não querê-las...\n", fptr);
        fputs("Que tristes os caminhos, se não fora\n", fptr);
        fputs("A presença distante das estrelas!\n", fptr);
        fclose (fptr);
        return 0;
}
```

Informações Complementares

Manipulação de arquivos em txt

Funções do Grupo 2

Para strings.

Lendo uma cadeia de caracteres de um arquivo:

fgets(str, qtd, Arquivo);

str = ponteiro char para a cadeia de caracteres em que os dados lidos do arquivo serão colocados

qtd = o número máximo de caracteres a serem lidos

Arquivo = ponteiro para a estrutura FILE do arquivo

Informações Complementares

Manipulação de arquivos em txt

Exemplo 6: leitura de um arquivo, uma linha por vez e impressão no monitor

```
#include<stdio.h>
#include<stdlib.h>

int main( void )
{
        FILE *fptr;
        char str[81];

        if((fptr = fopen("Testestr.txt","r")) == NULL){

                puts("Não foi possível abrir o arquivo");
                exit(1);
        }

        while (fgets(str,80,fptr) != NULL)
                printf("%s",str);

        fclose (fptr);

        return 0;
}
```

Informações Complementares

Manipulação de arquivos em txt

Funções do Grupo 3

Para dados formatados.

Informações Complementares

Manipulação de arquivos em txt

Funções do Grupo 3

Para dados formatados.

Gravando dados formatados no arquivo:

fprintf(FILE *ponteiro_arquivo, const char *formato, [argumentos]);

sendo const char *formato: “%d”, “%f”, “%s”, etc.

[argumentos] : nomes das variáveis envolvidas

Todas as possibilidades de formato de printf ( ) operam com fprintf ( ) .

Informações Complementares

Manipulação de arquivos em txt

Exemplo 7: escrevendo valores de 3 variáveis em um arquivo

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define TRUE 1

int main( void )
{
        FILE *fptr;
        char titulo[30];
        int regnum;
        double preco;

        fptr = fopen("livros.txt","w");

        while (TRUE){

        printf("\nDigite o titulo, registro e preco do livro: ");
        scanf("%s %d %lf", titulo, &regnum, &preco);
```

Informações Complementares

Manipulação de arquivos em txt

```
            if(strlen(titulo) <= 1) break;

            fprintf ( fptr, "%s %d %.2lf \n", titulo, regnum, preco);

    }

    fclose (fptr);

    return 0;
}
```

Em execução:

Digite titulo, registro e preco do livro: Helena 102 70.5

Digite titulo, registro e preco do livro: Iracema 321 63.25

Digite titulo, registro e preco do livro: Macunaima 156 73.8

Dados serão escritos no arquivo livros.txt

Informações Complementares

Manipulação de arquivos em txt

Funções do Grupo 3

Para dados formatados.

Lendo dados formatados no arquivo:

fscanf(FILE *ponteiro_arquivo, const char *formato, [argumentos]);

sendo const char *formato: “%d”, “%f”, “%s”, etc.

[argumentos] : nomes das variáveis envolvidas

Todas as possibilidades de formato de scanf ( ) operam com fscanf ( ) .

Informações Complementares

Manipulação de arquivos em txt

Exemplo 8: leitura de um arquivo com dados formatados

```
#include<stdio.h>
#include<stdlib.h>

int main( void )
{
        FILE *fptr;
        char titulo[30];
        int regnum;
        double preco;

        fptr = fopen("livros.txt","r");

        while ( fscanf(fptr, "%s %d %lf", titulo, &regnum, &preco) != EOF)
                printf("%s %d %.2lf \n",titulo, regnum, preco);

        fclose (fptr);

        return 0;
}
```

Informações Complementares para os Experimentos

Em linguagem C

Trabalhando com arquivos em txt

Trabalhando com arquivos: funções do Grupo 4:

Para gravar e ler blocos de bytes.

Gravando matrizes de bytes ou estruturas:

Função fwrite( ) .

```
fwrite(void *buffer,
 int tamanho_da_unidade,
 int numero_de_unidades,
 FILE *fp);
```

Lendo matrizes de bytes ou estruturas:

Função fread( ) .

```
fread( void *buffer,
 int tamanho_da_unidade,
 int numero_de_unidades,
 FILE *fp);
```

Exemplo 9: usando as funções fwrite( ) e fread( )

```
/* copiaarq.c */
/* Mostra o uso de fread() e fwrite() */
#include <stdio.h> /* Define FILE */
#include <stdlib.h>

void CopiaArquivo(const char* Origem, const char *Destino);

int main()
{
   char Origem[] = "copiaarq.c";
   char Destino[]= "copiaarq.cpy";

   CopiaArquivo(Origem, Destino);
   system("type copiaarq.cpy");
   system("pause");
   return 0;
}

void CopiaArquivo(const char* Origem, const char *Destino)
{
   unsigned char buffer[1024];
   int BytesLidos;

   FILE *fOrigem, *fDest; /* Ponteiro para arquivo */
```

Exemplo 9: usando as funções fwrite( ) e fread( )

```
    if((fOrigem = fopen(Origem,"rb"))==NULL) return;
    if((fDest = fopen(Destino,"wb"))== NULL) return;
    do
    {
       BytesLidos= fread(buffer,sizeof(char),sizeof(buffer), fOrigem);
       fwrite(buffer,sizeof(char), BytesLidos, fDest);
    } while(BytesLidos);

    fclose(fOrigem);
    fclose(fDest);

}
```

Exercício

Construir um tradutor, na forma de um programa em linguagem C completo, que avalie, em um arquivo txt, um programa fonte formatado em linguagem C, procurando encontrar lexemas e padrões de tokens. Caso encontre, deve informar a ocorrência criando 2 arquivos txt de saída: um com o fluxo de tokens e seus valores de atributo respectivos, e outro com a tabela de símbolos.

Há várias maneiras de fazer.

Uma delas está descrita à partir da página 12, item 2.1.1, na nossa apostila de Introdução aos Compiladores.

Continua...